

STORELINK

แพลตฟอร์มบริหารจัดการร้านค้า

ความเป็นมาและปัญหา

ปัจจุบันร้านค้าขนาดเล็กและขนาดกลางต้องจัดการงานหลายด้าน เช่น การขาย สต็อก พนักงาน และยอดขาย แต่กลับขาดระบบที่รวมข้อมูลทั้งหมดไว้ด้วยกัน ส่งผลให้เกิดความผิดพลาดจากการบันทึกด้วยมือ ข้อมูลกระจัดกระจาย และไม่สามารถวิเคราะห์ยอดขายได้อย่างแม่นยำ

StoreLink จึงถูกพัฒนาเพื่อเป็นแพลตฟอร์มที่รวมการทำงานทั้งหมดไว้ในระบบเดียว ช่วยให้เจ้าของร้านมองเห็นภาพรวมธุรกิจและบริหารจัดการได้ง่ายขึ้นจากทุกที่

วัตถุประสงค์ของโครงการ

โครงการนี้มุ่งพัฒนาระบบบริหารจัดการร้านค้าแบบครบวงจรที่รองรับหลายประเภทธุรกิจ พร้อมระบบจัดการสินค้า สต็อก พนักงาน และการขายแบบเรียลไทม์ รวมถึงฟังก์ชันสรุปยอดขายเพื่อช่วยให้ผู้ประกอบการวางแผนธุรกิจได้อย่างแม่นยำ นอกจากนี้ยังเน้นความง่ายในการใช้งาน ความถูกต้องของข้อมูล และความยืดหยุ่นในการขยายระบบในอนาคต

ขอบเขตของระบบ

StoreLink ออกแบบให้รองรับร้านค้าทั่วไป และร้านอาหาร โดยผู้ใช้งานสามารถเข้าถึงระบบผ่านเว็บได้จากทุกสถานที่ ภายในระบบประกอบด้วยหน้าจอสำหรับจัดการสินค้า สต็อก พนักงาน และการขาย รวมถึงรายงานยอดขายแบบเรียลไทม์ อีกทั้งยังรองรับการบริหารร้านค้าหลายแห่งภายใต้บัญชีเดียว ซึ่งข้อมูลของแต่ละร้านจะถูกแยกเก็บอย่างเป็นระบบ

ผู้ใช้งานของระบบ

Admin

- มีสิทธิ์ตรวจสอบข้อมูลทั้งหมด
- จัดการร้านค้า
- จัดการผู้ใช้

Owner

- สร้างและจัดการร้านค้า
- จัดการข้อมูลสินค้า
- จัดการข้อมูลพนักงาน
- สามารถดูรายงานยอดขาย

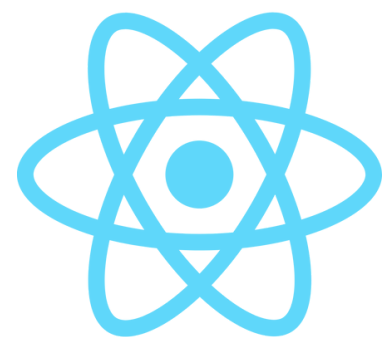
Employee

- ขายสินค้า
- ตรวจสอบสต็อก
- มีสิทธิ์เฉพาะหน้าที่ที่ได้รับมอบหมาย

เครื่องมือและเทคโนโลยีที่ใช้



เครื่องมือและเทคโนโลยีที่ใช้



FRONTEND



BACKEND



DATABASE

เครื่องมือและเทคโนโลยีที่ใช้

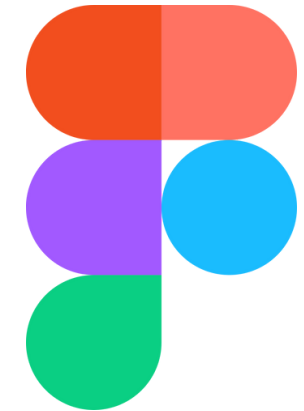


JWT

AUTHENTICATION



API TESTING



DESIGN



UPLOAD IMAGE



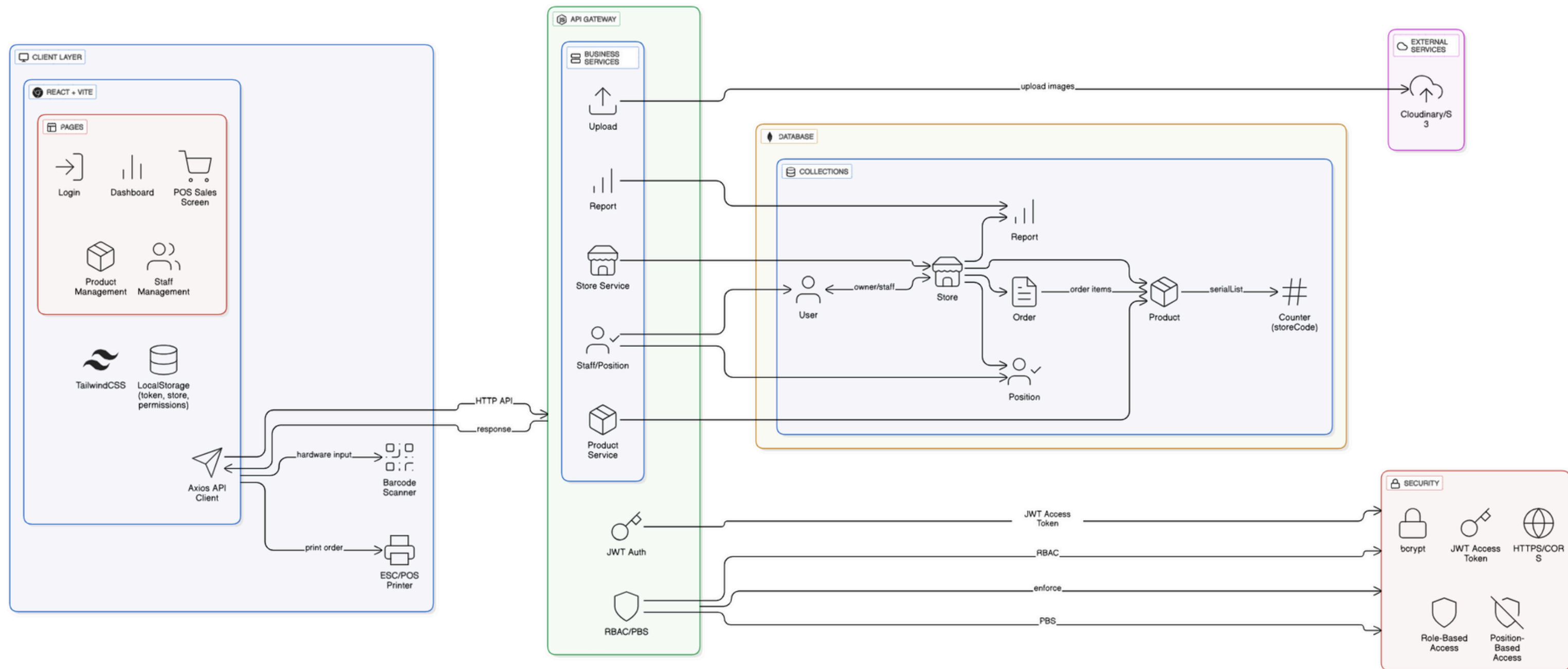
Vercel



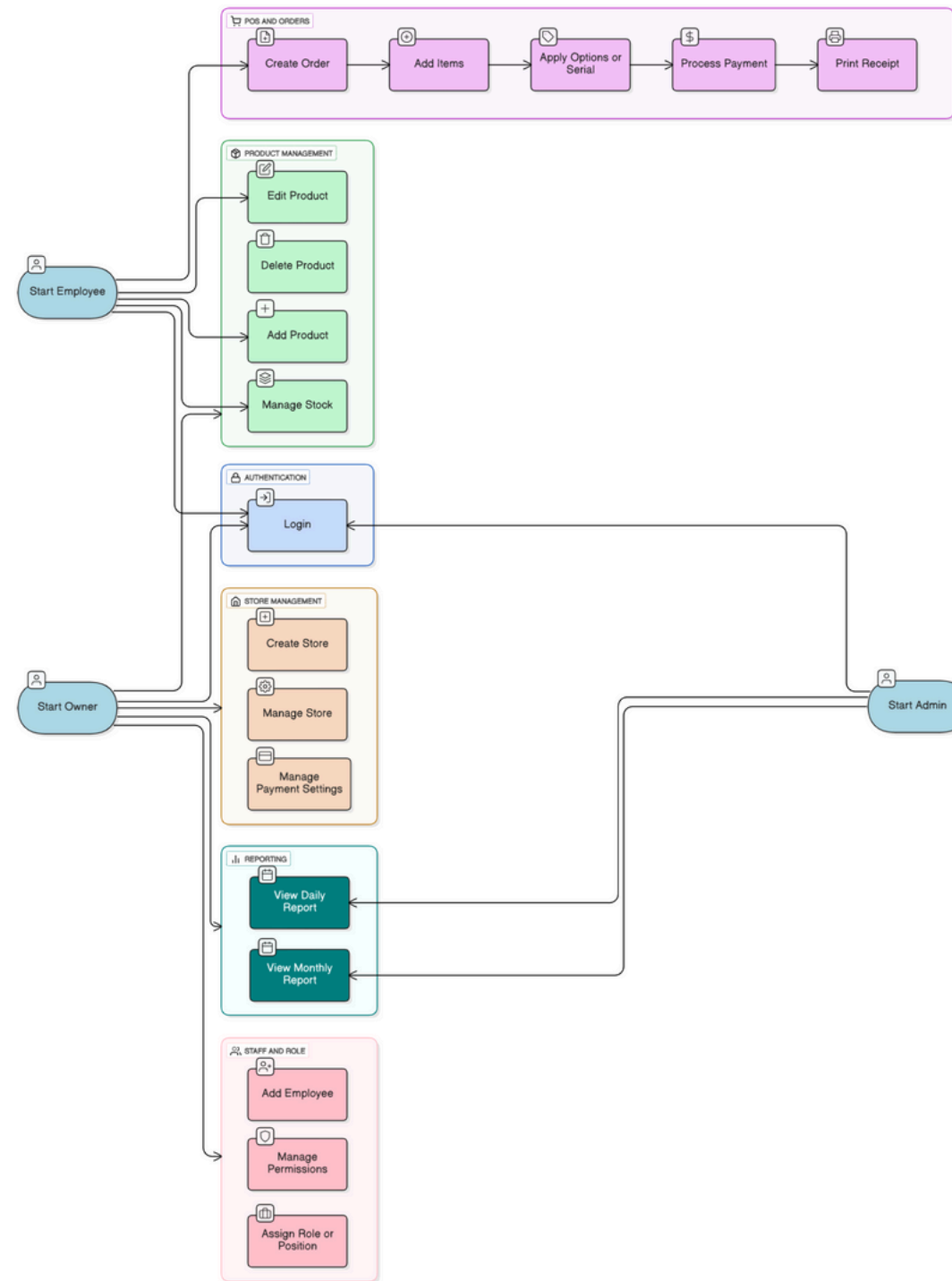
Render

DEPLOY

SYSTEM ARCHITECTURE



USE CASE



USER SCHEMA

```
const userSchema = new mongoose.Schema({
  firstName: { type: String, required: true },
  lastName: { type: String, required: true },
  gender: { type: String, enum: ["Male", "Female", "Other"], default: null },
},
  birthDate: { type: Date, default: null },
  idCard: { type: String, default: null },
  idCardImage: { type: String, default: null },
  phone: { type: String, default: null },
  email: { type: String, default: null },
  address: { type: String, default: null },
  emergencyContact: { type: String, default: null },
  username: { type: String, unique: true, sparse: true },
  password: { type: String, required: true },
  role: { type: String, enum: ["Admin", "Owner", "Employee"], default:
"Owner" },
  storeIds: [{ type: mongoose.Schema.Types.ObjectId, ref: "Store" }],
  positionId: { type: mongoose.Schema.Types.ObjectId, ref: "Position" },
  status: { type: String, enum: ["active", "inactive"], default: "active"
},
  hireDate: { type: Date, default: Date.now },
  resignDate: { type: Date, default: null },
  salary: { type: Number, default: null },
  note: { type: String, default: null },
}, { timestamps: true });
```

POSITION SCHEMA

```
const positionSchema = new mongoose.Schema({  
  storeId: { type: mongoose.Schema.Types.ObjectId, ref: "Store", required:  
true },  
  positionName: { type: String, required: true },  
  permissions: [{ type: String }]  
}, { timestamps: true });
```

STORE SCHEMA

```
const storeSchema = new mongoose.Schema(  
  {  
    storeCode: { type: String, unique: true },  
    ownerId: { type: mongoose.Schema.Types.ObjectId, ref: "User",  
required: true },  
    storeName: { type: String, required: true },  
    storeImage: { type: String, default: "" },  
    storeType: {  
      type: String,  
      enum: ["general", "restaurant", "service"],  
      required: true,  
    },  
    address: String,  
    phone: String,  
    taxRate: { type: Number, default: 0 },  
    paymentSettings: { type: paymentSettingsSchema, default: () => ({}),  
    },  
    { timestamps: true }  
  );
```

PRODUCT SCHEMA

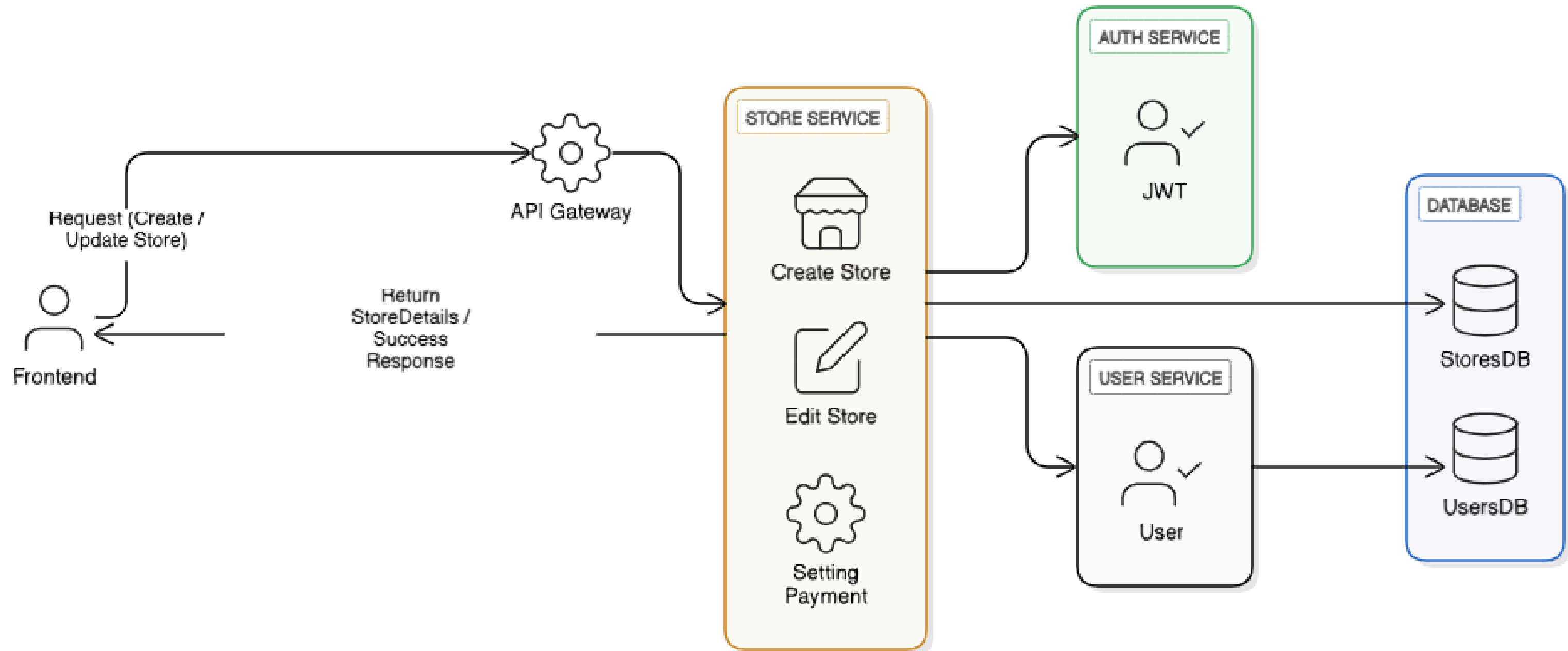
```
const productSchema = new mongoose.Schema({
  storeId: { type: mongoose.Schema.Types.ObjectId, ref: "Store",
required: true },
  barcode: { type: String, unique: true, sparse: true },
  name: { type: String, required: true },
  category: String,
  type: { type: String, enum: ["standard", "serialized"], default:
"standard" },
  price: { type: Number, required: true },
  unit: String,
  duration: { type: Number, default: 0 },
  stockQty: { type: Number, default: 0 },
```

```
  optionGroups: [
    {
      name: { type: String, required: true }, // e.g., "Sweetness",
"Toppings"
      selectionType: { type: String, enum: ["single", "multiple"],
default: "single" },
      required: { type: Boolean, default: false },
      maxSelections: { type: Number },
      choices: [
        {
          label: { type: String, required: true },
          value: { type: String },
          priceDelta: { type: Number, default: 0 },
          isDefault: { type: Boolean, default: false }
        }
      ]
    },
    {
      serialList: [
        {
          serialNumber: { type: String, unique: true, sparse: true },
          status: { type: String, enum: ["available", "sold"], default:
"available" }
        }
      ],
      status: { type: String, enum: ["available", "unavailable"], default:
"available" },
      description: String,
      productImage: { type: String, default: "" },
    }, { timestamps: true }]);
```

ORDER SCHEMA

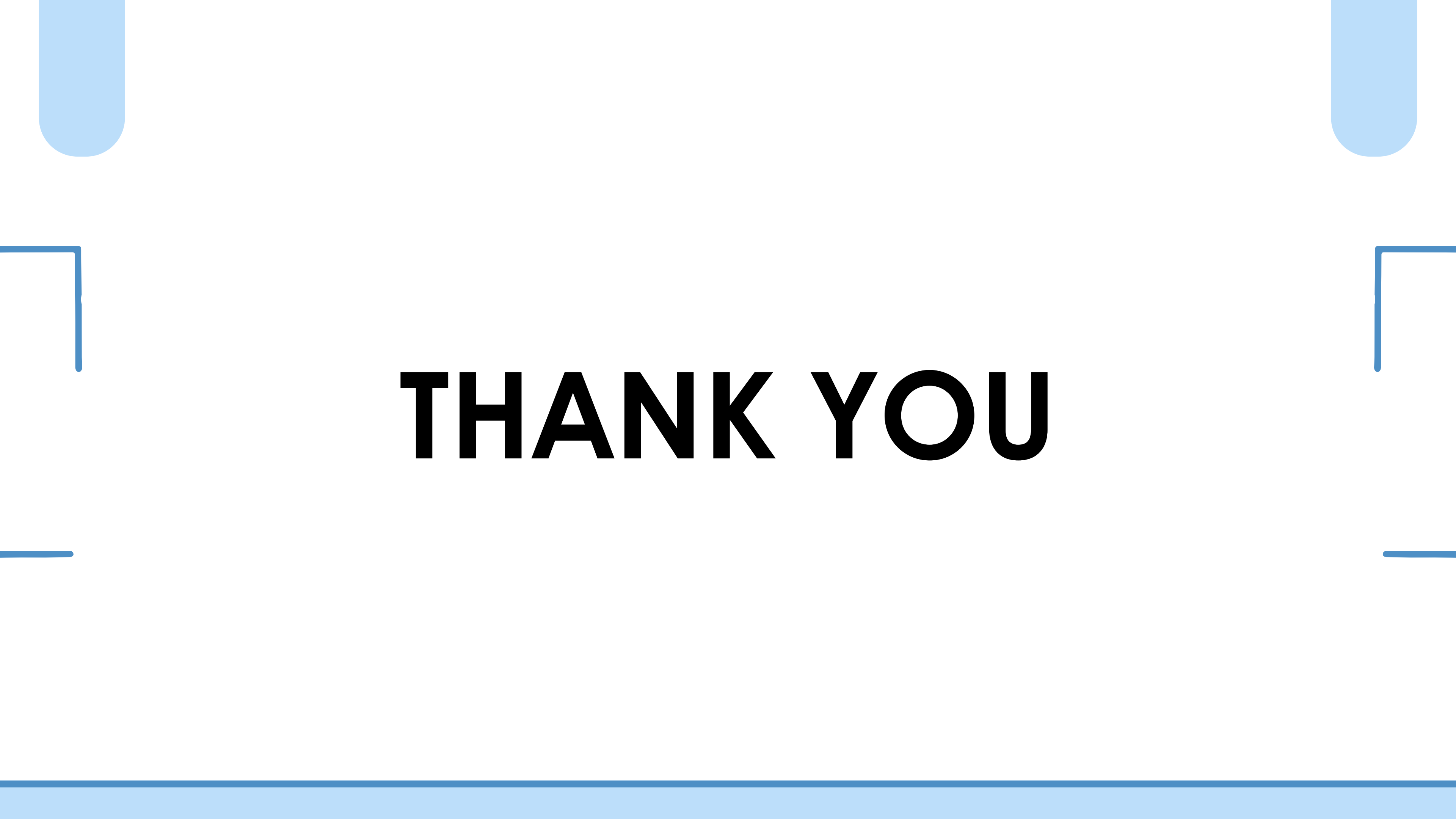
```
const orderSchema = new mongoose.Schema({
  storeId: { type: mongoose.Schema.Types.ObjectId, ref: "Store",
required: true },
  userId: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
  kitchenStatus: {
    type: String,
    enum: ["waiting", "cooking", "done"],
    default: "waiting"
  },
  items: [orderItemSchema],
  subTotal: Number,
  tax: Number,
  total: Number,
  paymentMethod: {
    type: String,
    enum: ["cash", "promptpay", "other", "none"],
    default: "none",
  },
  paidAmount: { type: Number, default: 0 },
  changeAmount: { type: Number, default: 0 },
  status: {
    type: String,
    enum: ["pending", "paid", "cancelled"],
    default: "pending",
  },
  receiptNumber: { type: String },
  paidAt: { type: Date },
  createdAt: { type: Date, default: Date.now },
  updatedAt: { type: Date, default: Date.now },
});
```

REPORT MICROSERVICES ARCHITECTURE DESIGN



REPORT API CHECKLIST

Admin จัดการข้อมูลร้านค้าสำหรับผู้ดูแลระบบ			^
GET	/admin/stores	ดึงข้อมูลร้านค้าทั้งหมด	  ✓
DELETE	/admin/stores/{id}	ลบร้านค้าออกจากระบบ	 ✓
Auth ระบบจัดการการเข้าสู่ระบบและบัญชีผู้ใช้			^
POST	/auth/register	สมัครสมาชิกใหม่	✓
POST	/auth/login	เข้าสู่ระบบ	✓
GET	/auth/profile	ดึงข้อมูลโปรไฟล์ผู้ใช้ปัจจุบัน	 ✓
Employees จัดการข้อมูลพนักงานในร้านค้า			^
GET	/employees/{storeId}	ดึงข้อมูลพนักงานทั้งหมดในร้าน	 ✓
POST	/employees/create/{storeId}	เพิ่มพนักงานใหม่ในร้าน	 ✓
PUT	/employees/update/{id}	แก้ไขข้อมูลพนักงาน	 ✓
DELETE	/employees/delete/{id}	ลบพนักงานออกจากระบบ	 ✓



THANK YOU